

Abstract— Adaptive rehabilitation requires a controller that keeps an exercise challenging without making it unattainable as user behavior changes. This paper presents an end-to-end reinforcement learning framework for a tabletop upper-limb pursuit task in which a UR10 robot drives an unpowered magnetic microrobot beneath an acrylic workspace and a participant attempts to catch it. The controller regulates the hand-microrobot distance around a platform-specific Zone of Proximal Development band. The framework combines decoupled randomization of pursuit intent and motor execution, a dual-stream encoder for current geometry and recent motion, self-supervised prediction of future hand displacement, and iterative league training with prioritized opponent sampling. In a ten-generation league without opponent identity, mean Time in Zone across learned hands increased from 0.188 for the initial robot to 0.282 for the final robot, while worst-hand Time in Zone increased from 0.138 to 0.204. In a separate representation ablation, the auxiliary GRU increased Time in Zone from 0.160 to 0.259 and reduced mean jerk from 0.165 to 0.127 relative to the feed-forward baseline. The final league policy also avoided the condition-specific collapse observed under scripted-only and single-opponent training. Nine zero-shot physical rollouts demonstrated online execution through the camera-to-robot loop and recovery toward the target band after transient changes in hand speed. These results show that behavior diversity and temporal motion encoding provide complementary mechanisms for robust adaptive difficulty regulation.

Index Terms— adaptive difficulty, domain randomization, league training, magnetic microrobot, reinforcement learning, robot-assisted rehabilitation, zero-shot transfer.

I. INTRODUCTION

Rehabilitation after neurological injury seeks to restore upper-limb motor function, which is essential for regaining independence in self-care, work, and everyday activities. Stroke and other neurological conditions can disrupt voluntary control, coordination, movement speed, and the ability to execute purposeful actions, often demanding extended rehabilitation. Repetitive, task-oriented practice is widely adopted to drive motor relearning, because it encourages users to actively generate movements, evaluate outcomes, and make continuous corrections. However, the effectiveness of such practice depends not only on how much training is performed but also on how difficult each trial feels. Tasks that are too easy may fail to provoke sufficient corrective effort, whereas overly difficult tasks can lead to repeated failure and disengagement. Effective upper-limb training should therefore sustain active participation while maintaining a challenge that feels demanding yet attainable. Striking this balance reliably is difficult, especially given the variability in motor performance across individuals and within a single training session.

In current rehabilitation practice, upper limb training is delivered through several complementary approaches. Therapist-guided exercises allow direct observation of movement quality and let therapists adjust task settings, assistance levels, and feedback for each person. Conventional tabletop devices and rehabilitation equipment offer accessible ways to repeatedly practice reaching, tracking, grasping, and coordination tasks. Robotic systems, including exoskeletons and end-effector devices, offer consistent task execution, programmable assistance or resistance, and numerical measurement of movement and force data. Virtual reality systems, motion-capture games, and screen-based exercises further expand training scenarios by adding visual feedback, scoring, and game-like progression. Together, these approaches address a wide range of rehabilitation needs, from personalized clinical guidance to intensive repetition, objective assessment, and task variety.

Despite their advantages, these approaches involve important compromises. Therapist-led training is highly adaptable but limited by therapist availability and the difficulty of scaling it up. Conventional devices are easy to use but often lack advanced sensing, real-time adaptation, or numerical feedback. Contact-based robotic systems can precisely control physical interaction but typically require users to wear or hold mechanical parts, which makes setup harder and limits how users can interact with the system. Fully virtual systems offer flexibility but mainly engage users through visual interfaces rather than through real physical interaction. To address these limits, we develop a system that lets users pursue a physical target within the workspace without needing to wear or hold any powered device. The target is moved without contact, and its behavior is defined in software, allowing flexible adjustment of paths, interaction patterns, and task difficulty. This design keeps direct physical interaction while reducing the equipment worn by the user and enabling programmable, adaptive training.

The effectiveness of this interface depends heavily on how the microrobot target is controlled during pursuit. The goal is neither to escape the user indefinitely nor to move so predictably that the task becomes trivial. Instead, the target motion should

keep the distance between hand and microrobot within a working Zone of Proximal Development (ZPD), a range that is challenging yet reachable. Achieving this requires continuous adjustment to changes in hand speed, movement direction, reaction delay, and pursuit strategy. Fixed paths and preset speeds cannot handle such variation. Controllers that react to distance can adjust motion based on current geometry, but their performance depends on parameters tuned by hand and may fail to account for recent motion trends. Moreover, actions that increase distance in the short term may lead to problems later, such as going out of bounds or making the task too hard in later steps. Reinforcement learning (RL) offers a suitable framework for this problem, since it can learn control policies that depend on the current state through interaction and can optimize long-term outcomes. RL has proven effective in robotic manipulation, locomotion, human-robot interaction, and adaptive rehabilitation control. However, applying RL to this task requires both a sufficiently varied set of simulated hand behaviors and a state representation able to capture motion dynamics over time.

Accordingly, we develop an adaptive RL framework that combines virtual hand randomization, a representation of motion over time, and training based on a population of policies. To widen the variety of simulated behaviors, pursuit strategy is separated from motor execution: scripted and learned hand policies generate the intended motion, while a separate execution layer adds variation such as inertia, limits on how fast motion can change, observation noise, and time delay. The robot policy captures the current interaction geometry along with recent relative hand motion, allowing it to represent both spatial layout and short-term dynamics together. An extra prediction objective is added to help the model learn motion dynamics. Robustness across pursuit strategies is further improved through repeated league training, in which the robot is trained against a growing pool of scripted and learned hand behaviors. The framework is tested through simulation experiments that assess robustness across pursuit behaviors and examine the contributions of time-based encoding and the extra prediction objective. The learned policy is then deployed, without further physical training, on the magnetic microrobot platform to test the full control loop from visual observation to physical actuation. The evaluation focuses on adaptive difficulty regulation, robustness, motion smoothness, system responsiveness, and the feasibility of physical deployment, rather than clinical effectiveness or motor recovery.

The rest of this paper is organized as follows. Section II introduces the magnetic tabletop platform and defines the adaptive pursuit control problem. Section III describes the virtual hand modeling approach, policy architecture, and training procedure. Section IV presents simulation experiments, ablation studies, and results from deploying the system on physical hardware without further training. Section V discusses the findings, limitations, and possible implications, and Section VI concludes the paper.

II. SYSTEM OVERVIEW AND PROBLEM FORMULATION

We introduce a magnetically actuated, noncontact rehabilitation system for active motor training of the upper limb. The user interacts with an unpowered magnetic microrobot on a tabletop workspace, while a robotic arm drives the microrobot indirectly through a permanent magnet placed below the surface. This layout preserves an active pursuit task while avoiding rigid physical coupling between the robot and the human body. The user acts as the pursuer rather than a passive follower, which supports active engagement in motor rehabilitation. Because an appropriate challenge level requires the microrobot to adapt continuously to the participant's changing pursuit behavior, we formulate its motion control as a reinforcement learning (RL) problem for regulating interaction difficulty.

A. Hardware Platform

The platform separates actuation, interaction, and perception into three physical layers. In the actuation layer, a UR10 robotic arm is placed beneath the workspace, and a permanent magnet is mounted on its end effector to drive the microrobot above. In the interaction layer, an acrylic sheet 5 mm thick supports the unpowered microrobot and forms a physical barrier between the robotic arm and the user. In the perception layer, an overhead RGB camera sends images to a host computer. The computer estimates the microrobot and hand positions and transmits target pose commands to the UR10 over Ethernet.

This layout emphasizes both interaction safety and hardware simplicity. Fig. 1 summarizes the workflow from simulation training to visual observation, policy inference, and physical magnetic actuation. The acrylic sheet provides a fixed physical separation between the participant and the robotic arm, such that the participant interacts only with the lightweight, unpowered microrobot on the tabletop workspace. The UR10 remains beneath the surface, with software motion limits and its standard

emergency stop available during operation. The system forms a vision-based closed loop in which an overhead RGB camera tracks the hand and microrobot, the learned policy generates motion commands from these observations, and the robotic arm actuates the microrobot through the permanent magnet.

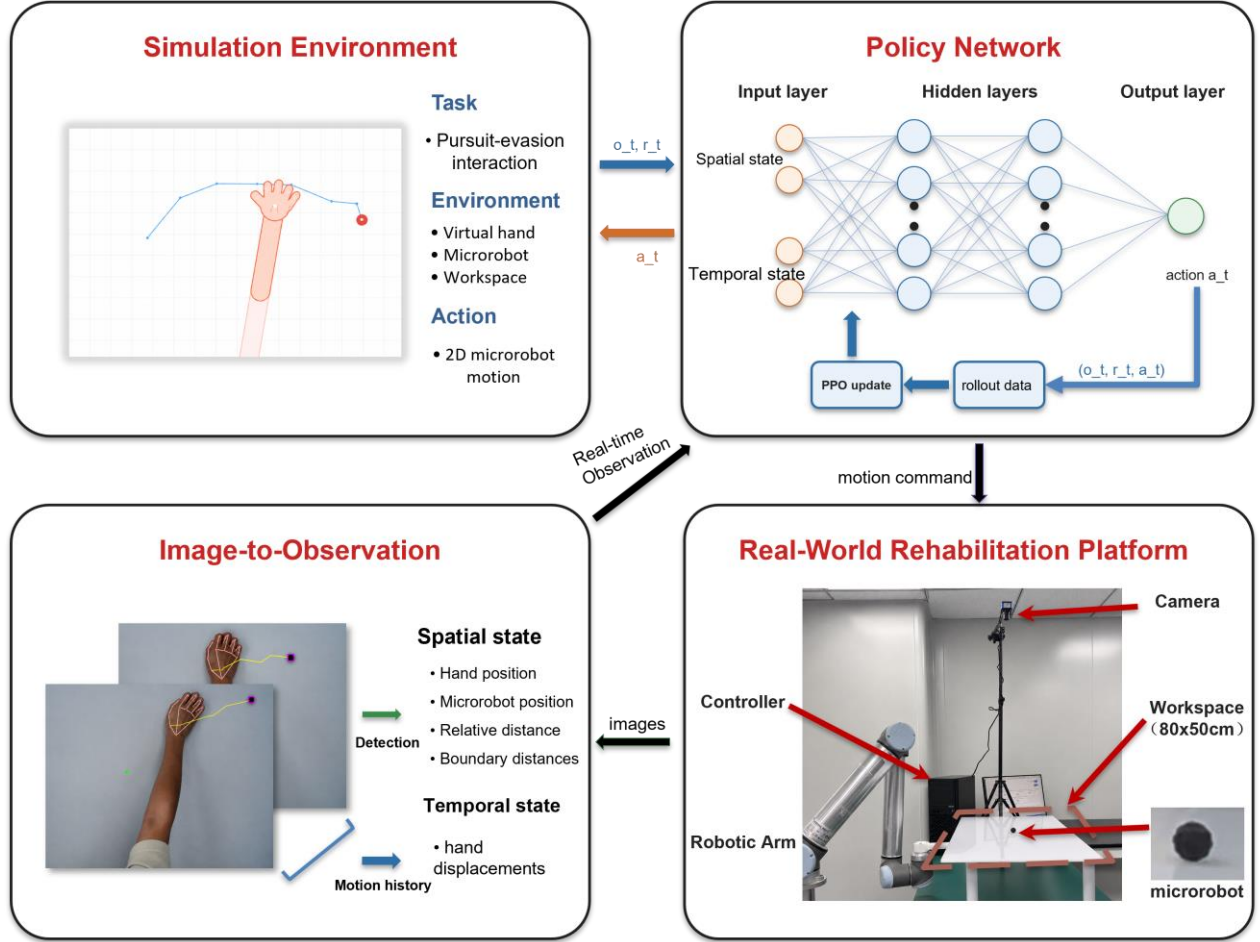


Fig. 1. System framework for noncontact rehabilitation from simulation to physical deployment. The simulation environment defines the pursuit interaction, the policy network maps spatial and temporal observations to microrobot actions, the observation module converts camera measurements into policy inputs, and the physical platform executes the learned policy through magnetic actuation.

B. Problem Formulation

We cast the task as a pursuit interaction with continuous feedback on a flat workspace. The participant's hand is the pursuer, and the microrobot is a moving target driven beneath the acrylic surface. The participant attempts to approach and catch the microrobot while the robot adjusts target motion to keep the task challenging but reachable. Unlike static reaching, the target continues to move after the participant initiates a movement. The task requires continuous tracking, anticipation, and correction rather than a single reaching movement.

The Zone of Proximal Development (ZPD) is defined operationally as a target range for the distance between the hand and microrobot. In this study, the target ZPD band was established after workspace calibration. It was selected for the desktop pilot platform to leave enough separation to avoid immediate catch while keeping the microrobot visually and physically reachable. This operating band is specific to the platform rather than a universal clinical threshold. Use with patients would require calibration during each session according to movement range, speed, and therapeutic goal. The robot should keep the interaction inside the target ZPD band rather than escaping completely or yielding passively. Hand speed, pursuit strategy, response delay, and motion noise can all change during interaction, making fixed difficulty settings inadequate.

Classical artificial potential field and rule-based reactive controllers provide intuitive solutions for regulating the motion of a target relative to a pursuer. However, their assumptions are not well matched to the present interactive pursuit task. A

distance-based potential field can drive the microrobot away from the hand, but it does not explicitly account for the direction, velocity, or recent motion pattern of the pursuing hand. When the participant changes from direct pursuit to interception, a purely reactive controller may respond only after the relative geometry has already changed. This delayed response can produce oscillatory motion, drive the microrobot toward the workspace boundary, or result in an early catch. More generally, a rule-based controller requires manually specified thresholds, gains, and motion patterns. Under camera delay, noisy hand estimates, and abrupt changes in pursuit strategy, fixed rules have limited ability to distinguish transient measurement variation from meaningful changes in hand behavior. These limitations motivate an RL policy that uses both current geometry and recent motion history to regulate interaction difficulty over a longer horizon.

For robot training, the reward is written as the sum of four intuitive terms: distance regulation, action smoothness, workspace safety, and catch avoidance. Let d_t be the distance between the hand and microrobot, a_t be the robot action, and $p_{R,t}$ be the microrobot position. The robot reward optimized by PPO is summarized as

$$r_t = r_{dist}(d_t) + r_{smooth}(a_t) + r_{bound}(p_{R,t}) + r_{catch}(d_t) \quad (1)$$

Equation (1) groups the robot reward into four components. The distance component $r_{dist}(d_t)$ encourages the separation between the hand and microrobot to remain inside the target ZPD band and penalizes deviations on either side. The smoothness component $r_{smooth}(a_t)$ discourages abrupt commanded motion. The boundary component $r_{bound}(p_{R,t})$ penalizes microrobot positions outside the valid workspace. The catch component $r_{catch}(d_t)$ penalizes contact between the hand and microrobot. Episodes terminate after catch, workspace exit, or the maximum horizon.

III. METHODOLOGY

The proposed framework is designed to improve robustness to variations in pursuit strategy and motor execution. It consists of three main components. First, decoupled domain randomization separates movement intent from motor execution constraints. Second, a dual-stream encoder combines instantaneous geometry with recent hand motion, while an auxiliary dynamics head encourages short-horizon motion prediction. Third, iterative league training exposes the robot to an expanding pool of hand policies, reducing overfitting to a single virtual user.

A. Decoupled Randomization of Cognitive and Motor Factors

To improve robustness to diverse hand behaviors, we apply domain randomization to both pursuit strategy and motor execution. Rather than perturbing the resulting hand trajectory as a single process, we separate intended motion generation from its execution. A hand policy first generates the intended displacement, and a motor execution layer then applies motion constraints, observation noise, and temporal delay. The hand policy is sampled from a pool containing the SPC and trained RL policies, representing direct pursuit and more anticipatory interception behaviors.

For the stochastic scripted pursuit controller, let ρ_{ep} denote the stride length sampled for each episode and q_t denote a planar unit direction. The intended displacement is

$$u_t^{SPC} = \rho_{ep} q_t \quad (2)$$

The stride ρ_{ep} is sampled once at episode reset and then held fixed. The direction q_t follows a pursuit rule with ϵ exploration. With probability $1 - \epsilon$, q_t points from the current hand position toward the microrobot. With probability ϵ , q_t is sampled uniformly from all planar unit directions. In the experiments, ρ_{ep} is sampled uniformly between 0.45 and 0.70, and ϵ is fixed at 0.05. Learned hand policies output the intended displacement directly, and the same motor execution model is applied afterward.

Let u_t denote the intended hand displacement per simulation step, m_t denote the executed displacement, and $\tilde{m}_t$ denote the filtered target displacement. The coefficient α controls temporal smoothing, with smaller values placing more weight on the preceding executed displacement. The filtered target is

$$\tilde{m}_t = \alpha u_t + (1 - \alpha) m_{t-1} \quad (3)$$

The change from the preceding executed displacement is then limited by its Euclidean norm,

$$m_t = m_{t-1} + \text{clip_norm}(\tilde{m}_t - m_{t-1}, c_{\max}) \quad (4)$$

Here, $\text{clip_norm}(z, c)$ returns z when its Euclidean norm does not exceed c and otherwise rescales z to have norm c . The scalar $c_{\max}$ therefore bounds the Euclidean norm of the change in executed displacement between consecutive simulation steps. The resulting per-step displacement updates the simulated hand position.

$$p_{H,t+1} = p_{H,t} + m_t \quad (5)$$

Equation (5) applies the executed displacement to the hand position. Delay and observation noise are introduced separately by the motor execution layer. Under a constant intended displacement, the filtered and executed increments converge to a constant value rather than decaying toward zero.

TABLE I: MOTOR RANDOMIZATION PARAMETERS

Parameter	Symbol	Range
Motion smoothing coefficient	alpha	0.7
Maximum inter-step displacement change	c_max	0.15
Gaussian observation noise	sigma	Uniform distribution over 0.01–0.08
Neural delay	delta	0–3 frames (0–375 ms)

Uniform distribution denotes sampling uniformly over the stated interval.

B. Dual-Stream Encoder with Auxiliary Dynamics Prediction

Domain randomization exposes the robot to hand motions with different strategies, delays, and execution characteristics. Under this variability, the instantaneous interaction state alone may not be sufficient to determine the hand’s motion trend. We therefore use a dual-stream encoder that combines the current geometric state with a recent history of hand motion. An auxiliary dynamics prediction task further encourages the shared representation to capture short-term motion patterns.

At each timestep, the observation vector has 44 dimensions and is divided into a scalar vector with 12 dimensions and a temporal buffer with 32 dimensions, as shown in (6).

$$o_t = [s_t; h_{t-T:t}] \quad (6)$$

The scalar component represents the instantaneous geometry of the interaction and is defined in (7).

$$s_t = [p_R; p_H; d(R, H); b_N, b_S, b_E, b_W; \text{stride}; a_{t-1}] \quad (7)$$

Here, p_R and p_H denote the 2D positions of the microrobot and the hand. $d(R, H)$ is their Euclidean distance. b_N , b_S , b_E , and b_W denote signed distances to the workspace boundaries. The variables stride and a_{t-1} represent the current movement step size and the previous robot action, respectively.

The temporal buffer $h_{t-T:t}$ contains the $T = 16$ most recent relative hand displacement vectors, forming a sequence with 32 dimensions. This sequence helps the encoder estimate motion direction, response delay, and oscillatory behavior that are unavailable from the current position alone. Relative displacements make the representation invariant to translation, so the recurrent stream focuses on how the hand is moving rather than where the interaction occurs in the workspace.

The scalar vector and temporal buffer are encoded in separate streams, as shown in Fig. 2. A multilayer perceptron (MLP) extracts geometric information from the scalar state, while a gated recurrent unit (GRU) processes the displacement sequence. The two stream outputs are concatenated and passed through a fusion MLP to produce the shared representation used by the policy and value heads.

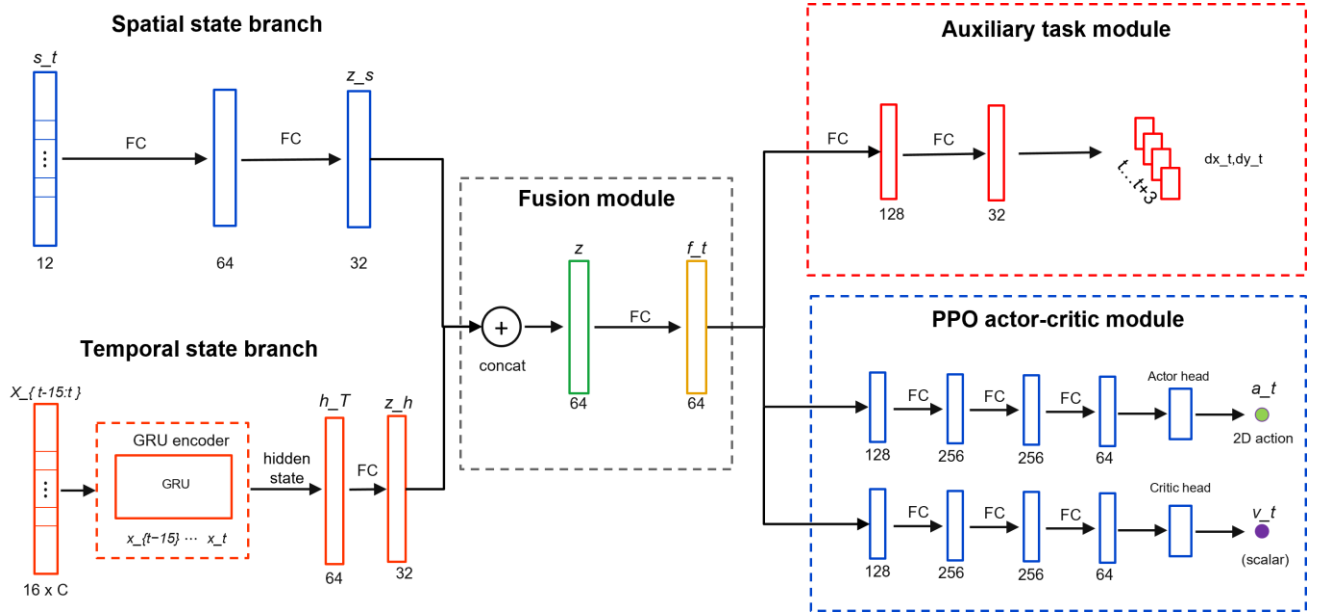


Fig. 2. Dual-stream policy architecture with auxiliary dynamics prediction. The spatial branch encodes the geometric state, the temporal branch encodes recent interaction history with a recurrent module, and the fused representation supports the policy head, value head, and auxiliary prediction head.

The ZPD reward provides the main control signal, but it supervises the encoder only indirectly through policy optimization. An auxiliary dynamics head adds a denser learning signal for motion representation. The head predicts relative hand displacement over a horizon of eight steps. Future displacements are available from the rollout, so this self-supervised auxiliary task requires no additional annotation.

The trajectory prediction loss is given in (8).

$$L_{traj} = E[\|\hat{D}_{t+1:t+H} - D_{t+1:t+H}\|_2^2], \quad H = 8 \quad (8)$$

Here, $\hat{D}_{t+1:t+H}$ is the predicted future displacement sequence, and $D_{t+1:t+H}$ is the observed future displacement sequence. The full training objective combines the PPO loss with the auxiliary trajectory prediction loss in (9).

$$L_{total} = L_{PPO} + \lambda_{traj} L_{traj} \quad (9)$$

The coefficient λ_{traj} controls the auxiliary loss weight. Because the auxiliary dynamics head shares the encoder with the policy, its gradients also update the spatial and temporal streams. These gradients encourage the encoder to capture motion inertia, delay, and interaction dynamics, helping the policy anticipate upcoming hand motion from recent history.

C. League Training

The preceding encoder addresses how recent hand motion is represented within an interaction. Generalization across pursuit behaviors also depends on which hand policies the robot encounters during training. A robot trained against a single hand policy may overspecialize and fail when the pursuit strategy changes. Simultaneous training of the robot and hand creates a different problem because both agents jointly determine the next state and return. Updating both policies together changes each agent's effective environment, violating the stationarity assumption used by PPO. In practice, the interaction can enter a cycle in which the robot exploits the current hand policy, the hand adapts, and the robot loses strategies that were useful against earlier hands.

The training schedule separates the two sides of learning and retains older hand behaviors in a shared pool. During robot training, sampled hand policies are fixed and treated as part of the environment. During hand training, the robot policy is fixed. The robot is trained against a pool P containing the SPC and learned hand policies from previous stages. This pool exposes the robot to direct pursuit as well as more strategic interception patterns. It broadens the distribution of hand behaviors without making the task purely adversarial, since the robot must maintain the ZPD interaction rather than simply maximize escape.

Algorithm 1 summarizes the iterative procedure. Each iteration first trains the robot against hands sampled from the fixed pool. It then trains a new hand policy against the frozen robot, initializes that hand from the previous learned hand when available, and adds the resulting policy to the pool for the next phase of robot training.

Algorithm 1: Iterative League Training	
Input:	initial robot policy $\pi_R^{(0)}$, scripted pursuit controller π_H^{SPC} , number of league iterations N .
Initialize:	pool of hand policies $P \leftarrow \{\pi_H^{SPC}\}$.
1. for	$n = 1, 2, \dots, N$ do
2.	Train robot policy $\pi_R^{(n)}$ with PPO against hand policies sampled from P .
3.	Initialize hand policy $\pi_H^{(n)}$ from $\pi_H^{(n-1)}$ when available.
4.	Train $\pi_H^{(n)}$ against the frozen robot policy $\pi_R^{(n)}$.
5.	Update pool of hand policies $P \leftarrow P \cup \{\pi_H^{(n)}\}$.
6. end for	

Uniform sampling from the pool of learned hands is inefficient because different hands challenge the current robot to different degrees. Recent episode length provides a simple competitiveness signal. Short episodes usually indicate that the hand quickly catches the microrobot or forces early termination, so the corresponding hand should be sampled more often during the next phase of robot training. For learned hand i , let ℓ_i be its recent mean episode length. Its sampling score is inversely related to this length and is smoothed with a uniform mixture.

$$s_i = \frac{1}{\ell_i + \epsilon_\ell}, \quad \tilde{p}_i = (1 - \mu) \frac{s_i}{\sum s_j} + \frac{\mu}{m} \quad (10)$$

Here, m is the number of learned hand policies, ϵ_ℓ is a small stabilizing constant, and μ controls the uniform exploration mass. The SPC remains in the pool with a fixed sampling probability, while the probability assigned to learned hands follows (10). This keeps the scripted pursuit behavior available and concentrates the remaining episodes of robot training on learned hands that still expose weaknesses in the current robot policy.

IV. EXPERIMENTS AND RESULTS

The evaluation combines simulation experiments with a physical deployment test. The simulation experiments measure robustness across hand behaviors and isolate the contribution of temporal encoding of hand history. The physical deployment test evaluates whether the learned policy can run online within the perception and control stack of the physical robotic platform and whether the realized motion responds coherently to measured hand movement.

A. Simulation Protocol and Metrics

The simulation study separates two sources of performance gain in the proposed framework. The league analysis tests whether expanding the pool of hand policies improves robustness across pursuit behaviors. The ablation analysis tests whether temporal encoding of hand motion and the auxiliary dynamics objective improve the learned robot representation. Both analyses use the simulation environment described in Section II, with the robot controlling the microrobot and the hand motion generated by the behavior models described in Section III.

All simulation evaluations used the same planar workspace and the target ZPD band defined in Section II.B. A timestep was counted as within the zone when the distance between the hand and microrobot fell inside the target ZPD band. The simulator and the physical workspace share the same calibrated coordinate convention, so one workspace unit corresponds to 1 cm after calibration and can be compared directly with physical rollout distances. The physical rollout in Section IV.D used the same target ZPD band for zero-shot consistency, not to suggest that this operating setting is clinically optimal for every user.

Time in Zone (TIZ) is the primary metric. It measures the fraction of the full episode horizon for which the distance between the hand and microrobot remains inside the target ZPD band. Episode length is reported in simulation steps and measures how long the interaction remains active before catch, workspace exit, or the maximum horizon. For the league analysis, robustness across learned hand policies is summarized by mean TIZ and by the lowest TIZ among the tested hands. For the network ablation, TIZ is complemented by workspace coverage and mean jerk, which respectively measure task coverage and motion smoothness in the final policy evaluation.

The comparison of training protocols uses an SPC baseline trained only against the stochastic scripted pursuit controller, a single policy baseline trained against one learned hand policy, and a league policy trained through iterative expansion of the hand pool. The robot policies are evaluated across learned hand generations in Fig. 3 and under three representative pursuit conditions in Table II. The representation comparison keeps the training setting fixed while changing the policy encoder. The MLP baseline uses instantaneous geometry, the GRU policy adds recent relative hand displacement, and the auxiliary GRU policy adds the objective for future dynamics prediction.

B. League Training and Robustness Across Hand Policies

The league evaluation asks whether an expanding pool of hand policies improves robustness across pursuit behaviors, rather than only improving performance against the most recent hand policy. After each robot generation is trained, it is evaluated against all learned hand generations in the pool. This separates generalization across hand behaviors from progress against a single training opponent. In Fig. 3(a), each row fixes one robot generation and each column fixes one learned hand generation. Early robot generations perform well only for limited parts of the hand pool, whereas later generations maintain stronger TIZ over a broader set of learned hands, suggesting reduced specialization. Fig. 3(b) summarizes the same matrix through mean TIZ and the lowest TIZ across tested hands. The mean curve reflects typical performance, and the lower curve tracks the most difficult learned hand for each robot generation. Both improve from early to final generations despite local fluctuations.

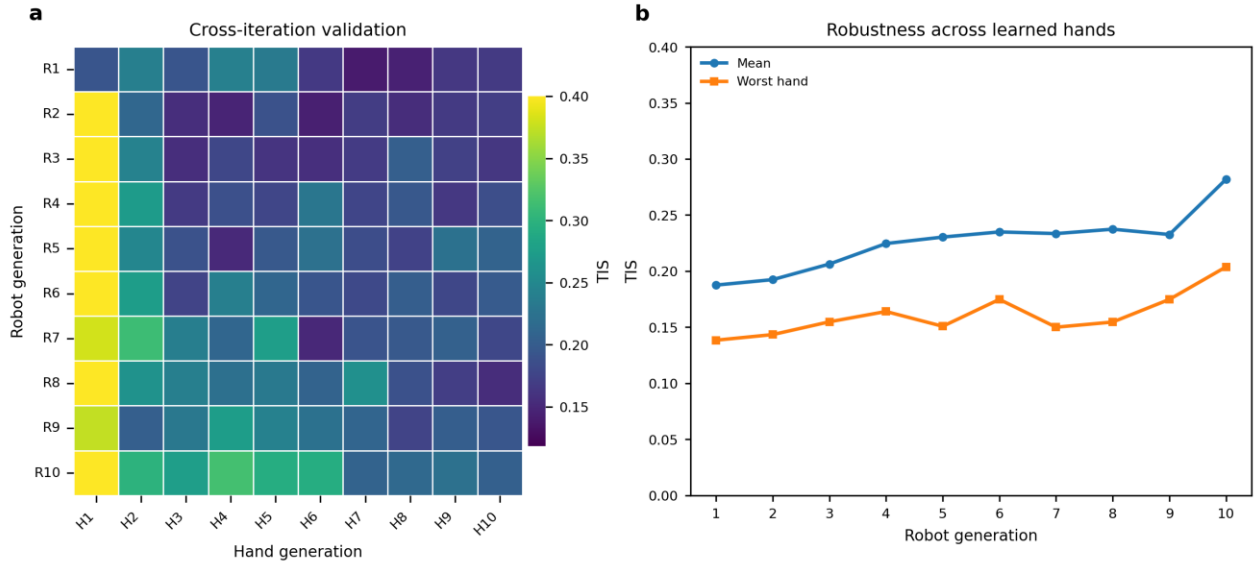


Fig. 3. Validation of robot policies trained through league across learned hand generations using the target ZPD band. (a) TIZ matrix obtained by evaluating each robot generation against each learned hand generation. Rows correspond to robot generations, and columns correspond to learned hand generations. (b) Mean TIZ and lowest TIZ across the learned hand test policies for each robot generation.

Table II extends the cross-generation analysis in Fig. 3 by comparing the SPC-only baseline, the single-opponent baseline, and the final league policy under SPC, learned-hand, and manual mouse pursuit. The learned-hand condition uses H1, the hand policy obtained after the first hand-training phase of the league procedure. The same H1 checkpoint is used to evaluate all three robot policies. H1 is also the sole training opponent for the single-opponent baseline and one member of the historical hand pool used for league training. For the automated SPC and H1 conditions, results are reported as the mean $\pm$ sample standard deviation across ten independently seeded evaluation trials, with 100 episodes per trial. The manual mouse condition was completed by one healthy operator who used the mouse interface to pursue and catch the simulated microrobot as quickly as possible. This condition introduces human reaction time and voluntary pursuit behavior but does not reproduce the biomechanics or perception noise of physical hand tracking. It is therefore treated as a human-in-the-loop stress test rather than clinical validation. Higher TIZ and longer episodes indicate that difficulty regulation was sustained for a larger portion of the interaction.

TABLE II: POLICY COMPARISON ACROSS THREE PURSUIT CONDITIONS

Robot policy	Pursuit condition	TIZ	Episode length (steps)
SPC-only baseline	SPC	0.31 ± 0.06	52 ± 8
SPC-only baseline	Learned hand H1	0.19 ± 0.04	31 ± 5
SPC-only baseline	Manual mouse	0.14 ± 0.03	33 ± 9
Single-opponent baseline	SPC	0.11 ± 0.02	25 ± 4
Single-opponent baseline	Learned hand H1	0.49 ± 0.07	70 ± 7
Single-opponent baseline	Manual mouse	0.19 ± 0.05	40 ± 11
League policy	SPC	0.38 ± 0.06	67 ± 8
League policy	Learned hand H1	0.48 ± 0.07	64 ± 8
League policy	Manual mouse	0.49 ± 0.09	74 ± 13

The two nonleague baselines exhibit clear condition-specific specialization. The SPC-only baseline performs relatively well under SPC pursuit but deteriorates under H1 and manual mouse pursuit. Conversely, the single-opponent baseline achieves its strongest result against its H1 training opponent but performs poorly under SPC and manual pursuit. The league policy avoids this narrow specialization. Although it does not exceed the single-opponent baseline under H1, it maintains strong performance under both automated pursuit conditions and achieves the highest TIZ and episode length in the manual mouse test. These results indicate that training against a broader distribution of hand policies produces more consistent difficulty regulation across scripted, learned, and human-controlled pursuit behaviors.

C. Network Ablation and Auxiliary Dynamics Analysis

The league results evaluate the diversity of hand behaviors used during training, while the network ablation tests whether the robot policy can use temporal information about hand motion once that diversity is present. The MLP baseline uses the instantaneous geometric state only, the GRU policy adds a history of relative displacement over 16 frames, and the auxiliary GRU further adds the dynamics prediction head described in Section III.B. The training curves in Fig. 4(a) and Fig. 4(b) show that temporal history improves optimization and interaction persistence. The GRU policies reach higher episode reward and longer episode length than the MLP baseline, indicating that current geometry alone is insufficient for stable difficulty regulation when hand motion changes over time. The auxiliary GRU gives the strongest training curve, suggesting that the prediction loss helps the recurrent stream form a more useful motion representation.

The auxiliary prediction examples in Fig. 4(c) provide a qualitative check on this representation. The predicted displacement over the short horizon generally follows the direction and curvature of the observed future hand motion rather than only reconstructing the current position. The final normalized metrics in Fig. 4(d) summarize the control effect of these representation changes. Recurrent history improves time within the ZPD and workspace coverage relative to the MLP baseline. Adding the auxiliary prediction objective further improves these performance metrics while reducing mean jerk. Together, the ablation indicates that temporal hand history and auxiliary dynamics prediction contribute to smoother and more reliable difficulty regulation.

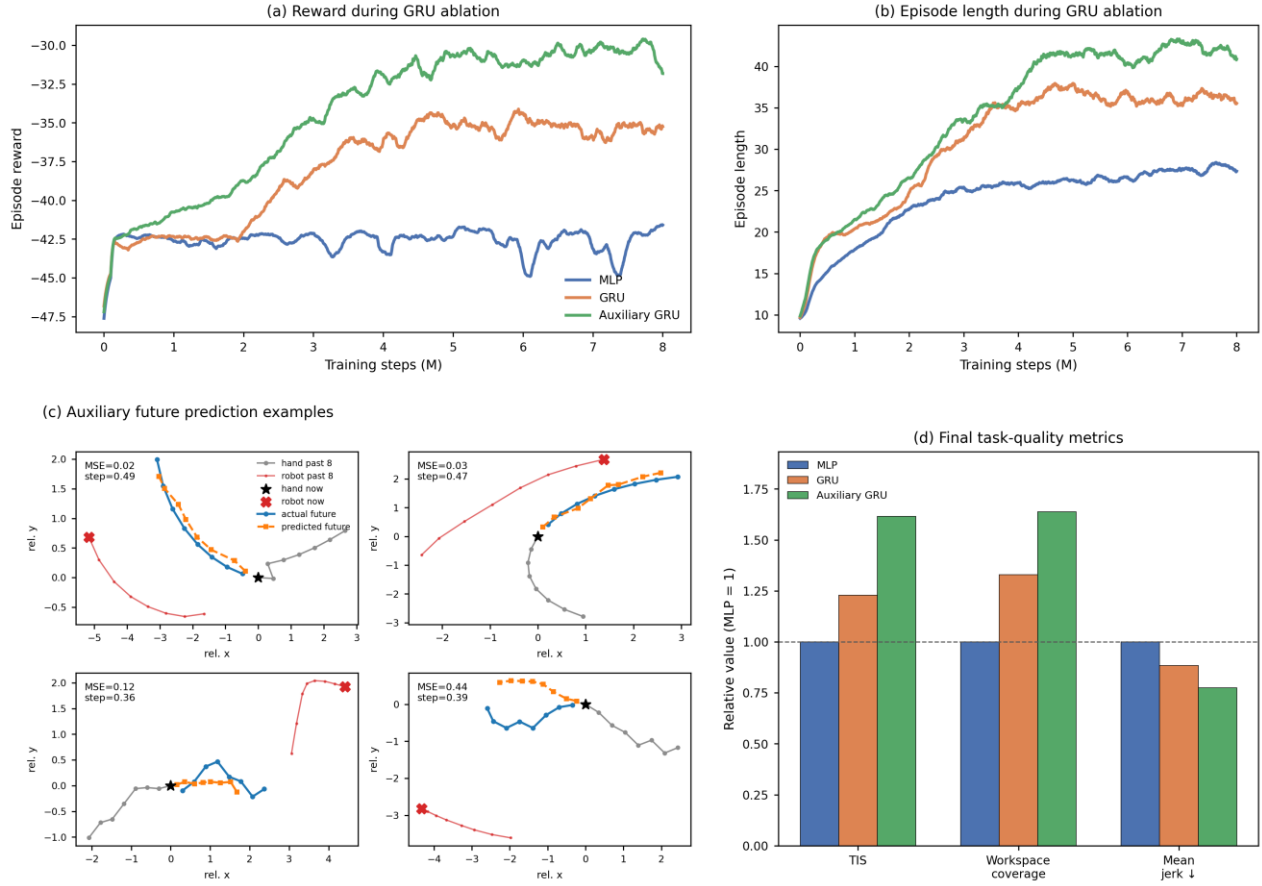


Fig. 4. Network ablation for temporal encoding and auxiliary dynamics prediction. (a) Smoothed episode reward during training for the MLP, GRU, and auxiliary GRU policies. (b) Smoothed episode length during training for the same policies. (c) Representative predictions from the auxiliary head over a short horizon compared with observed future hand motion. (d) Final performance metrics normalized to the MLP baseline; higher values are better for time within the ZPD and workspace coverage, whereas lower values are better for mean jerk.

D. Online Physical Deployment

To evaluate zero-shot transfer with real hand input, the simulation-trained policy was deployed on the physical microrobot platform without additional training. During each rollout, a healthy pilot pursued the passive microrobot across the acrylic workspace while the robotic arm moved the magnetic actuator beneath the surface. Nine rollouts were recorded during the deployment experiment. Camera observations and robot commands were processed online, allowing the policy to regulate microrobot motion under continuous feedback.

The deployment system retained the observation structure used in simulation. Calibrated camera measurements provided the hand and microrobot positions, while workspace-boundary features, the previously executed robot command, and the recent history of relative hand displacement completed the policy input defined in (6) and (7). Robot state measurements were used only for command execution and system monitoring and were not included in the policy observation. The perception pipeline combined YOLO-based microrobot detection, MediaPipe hand tracking, and a planar homography that mapped image coordinates to the workspace. When hand tracking was briefly interrupted, the controller extrapolated hand motion from the most recent velocity estimate. Safety was enforced through workspace limits, bounded command increments, automatic stopping after capture, controlled stopping during shutdown or interruption, and a hardware emergency stop.

Fig. 5 shows a representative zero-shot rollout with image frames and motion traces aligned in time. During steady pursuit, the hand-microrobot distance remains near the target ZPD band. A sharp increase in hand speed briefly reduces the separation below the target range. The policy responds by increasing microrobot speed, and the distance returns to the ZPD band over the following seconds. When the hand later slows, microrobot speed decreases accordingly and the separation again approaches the target range. Similar coupled responses appear throughout the rollout, showing that the policy adjusts microrobot motion to

changes in pursuit dynamics rather than applying a fixed-speed escape command. The final decrease in distance corresponds to capture and episode termination. Overall, the rollout shows online adaptation to abrupt changes in hand motion and recovery toward the intended interaction range after transient deviations.

Zero-shot physical deployment of the simulation-trained policy

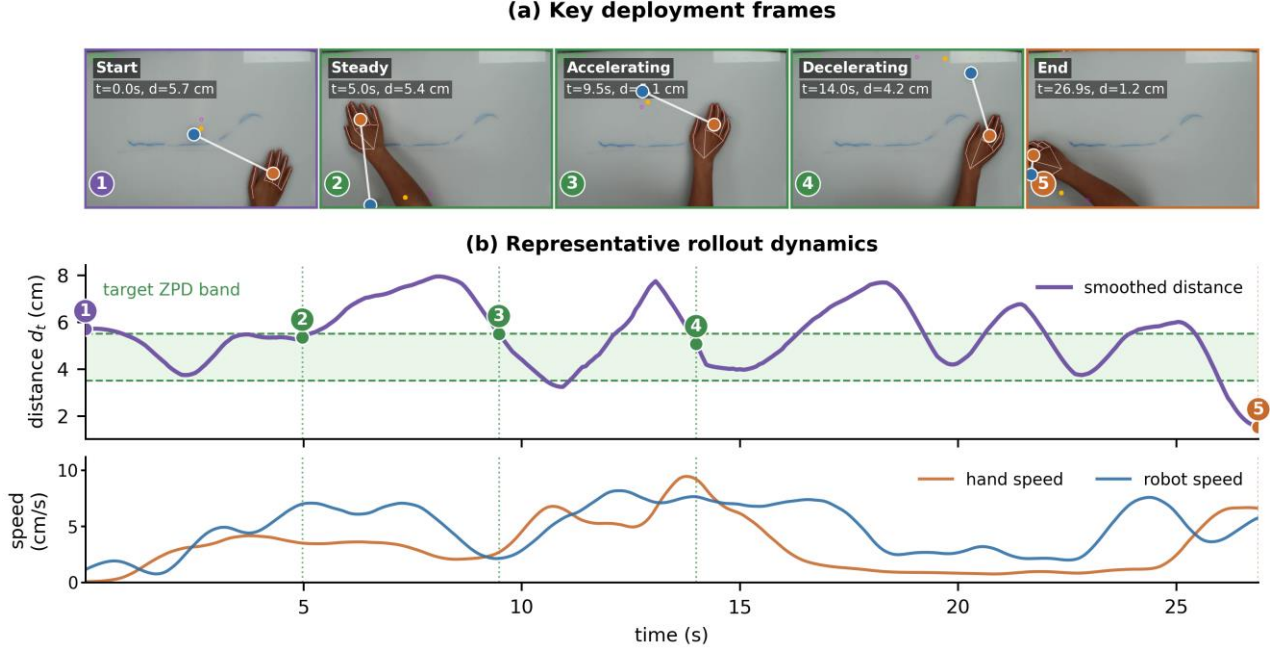


Fig. 5. Zero-shot physical deployment on the microrobot platform. (a) Representative frames from one of nine physical rollouts. (b) Smoothed hand-microrobot distance, target ZPD band, and measured hand and microrobot speeds from the same rollout.

V. DISCUSSION

On this platform, task difficulty is set by how the target moves rather than by any direct mechanical assistance to the limb, so the controller must keep adjusting the pursuit challenge while keeping the microrobot within reach. The simulation results suggest that this regulation depends on information at two different scales. Within a single episode, recent relative motion helps the policy read how the interaction between hand and microrobot is developing. Across training, exposure to varied pursuit strategies and execution styles reduces how much the policy depends on any one virtual hand. The ablation and league experiments support these two roles respectively. Zero-shot deployment further shows that the learned policy can run inside a real vision and control loop and change target motion in response to measured shifts in hand speed. Together these results support adaptive target motion as a workable way to regulate a noncontact physical task, though the evidence so far speaks only to control robustness, online responsiveness, and system feasibility.

At any given distance between hand and microrobot, the right robot response can differ a great deal depending on how that distance came about. The hand may be speeding up toward the target, slowing down after a failed approach, changing direction, or reacting after some delay. None of these differences show up directly if the policy sees only the current geometric state. Encoding recent relative displacement adds information about which way the hand is moving and how the situation is unfolding. This matches the ablation results, where Time in Zone rose from 0.161 for the MLP baseline to 0.197 with the GRU, then to 0.259 once auxiliary dynamics prediction was added. Workspace coverage improved along the same progression, while mean jerk went down. The auxiliary objective seems to strengthen the temporal representation by pushing the shared encoder to keep information useful for predicting hand motion a short time ahead. Prediction is not the end goal in itself; it simply gives a richer training signal for learning motion features that help with control. Policy performance therefore remains the main criterion, since predicting motion accurately does not by itself guarantee good regulation of task difficulty.

A second source of variability comes from differences between pursuit behaviors themselves. The scripted controller mostly produces direct pursuit, while learned hand policies can develop other patterns of interception and response. By separating how a strategy is generated from how it is executed, we can vary these behaviors further through inertia, limits on how fast motion can change, observation noise, and delay. League training widens the range of strategies further still, adding new hand policies while keeping earlier ones in the training pool. Performance across generations was not strictly monotonic, which reflects how the training distribution keeps shifting as robot and hand optimization alternate. Even so, mean Time in Zone across learned hands rose from 0.188 for the first robot generation to 0.282 for the final one, and the score against the toughest hand rose from 0.138 to 0.204. Comparing across individual conditions tells a similar story. A robot trained against a single learned hand did well against that closely matched opponent but got worse against scripted pursuit, while the league-trained policy stayed more balanced across scripted, learned, and manual pursuit. The main benefit of league training, then, is less specialization rather than the single highest score against any one behavior.

Moving the policy from simulation onto the UR10 platform brought in measurement noise, brief losses of tracking, communication delay, actuator dynamics, calibration error, and uncertainty in the magnetic drive. The deployment experiment tested all of this within the full loop of overhead tracking, coordinate transformation, building the temporal observation, policy inference, sending Cartesian commands, and magnetic actuation. Short gaps in hand detection were handled by extrapolating the most recent hand state from its estimated velocity, rather than pausing control right away. Nine physical rollouts were completed with no further training of the policy. In the representative rollout, a rapid increase in hand speed first reduced the distance to the microrobot. The robot then sped up, and the distance moved back toward the target ZPD band. When the hand later slowed down, the microrobot slowed down too. This paired response suggests the deployed policy was not simply running a fixed escape behavior at constant speed. Even so, the experiment only offers preliminary evidence of online responsiveness and zero-shot closed-loop operation; it does not establish clinical safety, therapeutic value, or an advantage over other rehabilitation interfaces.

Several factors limit how far these findings can be generalized. The simulated execution layer represents inertia, motion limits, observation noise, and delay through simplified processes, not parameters measured from people with motor impairment. It widens the training distribution, but it should not be read as a biomechanical or neurological model of a patient population. The target ZPD band was also chosen for the current desktop platform and kept the same across simulation and deployment for consistency. How suitable it is likely depends on reachable workspace, movement speed, response delay, fatigue, and the goal of a particular session. The physical platform is also still a proof of concept rather than a device ready for deployment. The UR10 gives accurate and flexible actuation for validating the control framework, but its size, cost, and installation needs make it unsuitable for routine use in hospitals or community rehabilitation settings. A practical system will need a smaller and cheaper actuation mechanism, along with tighter integration of the camera, computing unit, magnetic drive, and tabletop workspace. The physical evaluation itself was limited to nine rollouts from one healthy pilot participant. Manual mouse pursuit brought in voluntary human decisions and reaction time, but it did not reproduce impaired hand biomechanics or the full uncertainty that comes with tracking a hand visually. No study across repeated sessions, patient evaluation, or physical comparison against controllers that move at a fixed speed or react only to distance was carried out. Claims about engagement, motor learning, functional recovery, and clinical benefit therefore remain outside the scope of the current results.

The next step is to test the controller under broader and more controlled physical conditions. Studies with more healthy participants across repeated sessions should compare policies that move at a fixed speed, that react to distance alone, that are trained against a single opponent, and that are trained through the league, all under matched tasks. Relevant outcomes include how long it takes to return to the target ZPD band after a sudden change in hand speed, sensitivity to tracking interruptions, smoothness of commands, boundary violations, and variation between users. Recorded trajectories could then be used to estimate how each person tends to move and to calibrate the target band to their movement ability and to the goals of that session. At the same time, the current robot arm prototype should be replaced with a compact, flat magnetic drive able to reproduce the required target motion in a much smaller and cheaper form. Bringing the actuator, camera, controller, and workspace together into a single self-contained device would cut down on setup and make the system easier to use in hospitals and community settings. Testing additional movement directions and target behaviors would also help show whether the learned regulation holds up beyond the present pursuit task. Only after these control and usability questions have been settled should evaluation move to patient participants, with attention to tolerance, fatigue, engagement, movement quality, and

whether gains are retained across repeated sessions. Longer, longitudinal studies would ultimately be needed before any conclusions about rehabilitation outcomes can be drawn.

VI. CONCLUSION

This paper presented an end-to-end reinforcement learning framework for adaptive difficulty regulation in a magnetically actuated upper-limb pursuit task. Decoupled randomization separates pursuit intent from motor execution, the dual-stream encoder combines current geometry with recent hand motion, auxiliary prediction shapes the temporal representation, and iterative league training exposes the robot to a growing population of pursuit policies. The final league generation improved both mean and worst-hand Time in Zone across learned opponents and produced the most balanced results across scripted, learned, and manual pursuit.

Zero-shot deployment on the physical microrobot platform further showed that the simulation-trained policy can operate online with vision-based hand input and respond to changing pursuit speed. The framework therefore provides a basis for noncontact, data-driven rehabilitation tasks whose difficulty is regulated through interaction rather than fixed trajectories. Patient-calibrated Zone of Proximal Development settings and broader physical evaluation are the next steps toward clinical validation.